



深圳大学
Shenzhen University

互联网编程

第1讲

基本网络概念

深圳大学计算机与软件学院

主要内容

1. 基础概念
2. 互联网通信技术基础知识
3. 互联网通信应用程序



应用背景

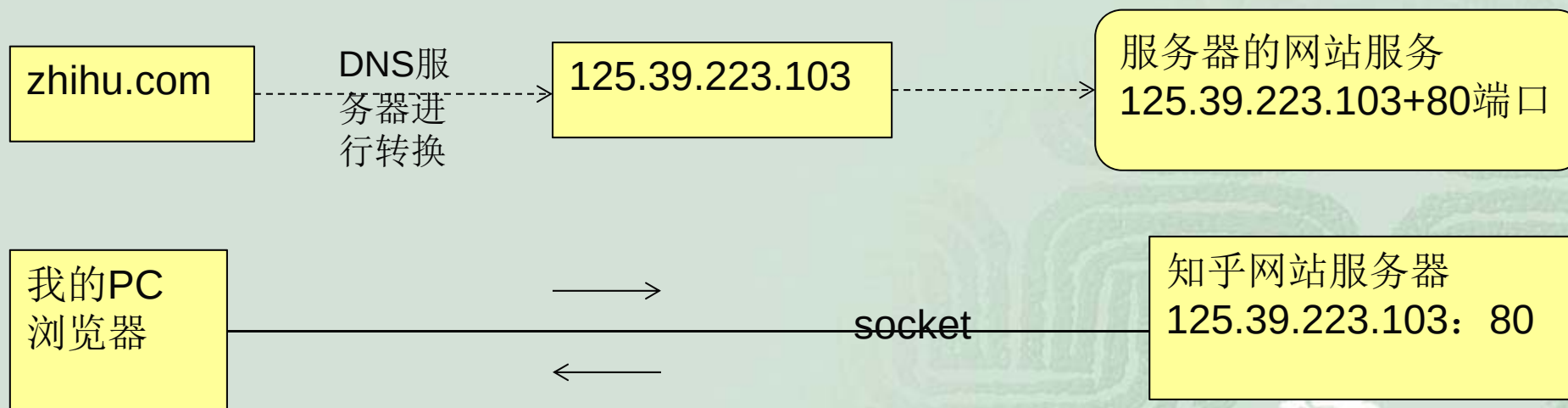
- Internet时代
- 各种互联网应用：如 网站、基于web的应用系统、...?



问题从访问一个网页开始

➤ 网站开发

➤ 我要访问一个网页，在浏览器中输入zhihu.com



易混淆的几个概念

- **Network**: 是最广的, 是汉语里面最广义的“网络”, 如各种电网、神经网络等。
- **互联网**: 因特网和其他类似的由计算机相互连接而成的大型计算机网络系统都是互联网, **Internet**是全球互联网中最大的一个。
- **Internet**: 中文直译因特网, 是由于许多小的网络(子网)互联而成的一个广域网, 每个子网中连接着若干台计算机(主机)。主要由计算机和电缆组成。因为 **Internet**是互联网中最大的一个, 所以也常将**Internet**称为互联网。
- **Web**: **www(world wide web)**, 万维网。它使用超文本技术将遍布全球的各种信息资源链接起来, 以便于用户访问。**web**是**Internet**上的一个应用层服务, 除了**web**外, **QQ**、**E-mail**等也是**Internet**上其他类型的应用层服务。

计算机网络(**computer network**)

- 节点/node: 计算机、打印机、路由器、网关等
- 主机/host: 计算机~~节点~~
- 地址/address: IPv4、IPv6、MAC
- 交换: 现代计算机网络都是包交换网络
- 协议/protocol: 通讯规则



网络应用程序(Network Application)

- 网络应用程序(Network Application)是指能够为网络用户提供各种服务的程序，它用于提供或获取网络上的共享资源。
- 如浏览程序、传输程序、远程登录程序等。



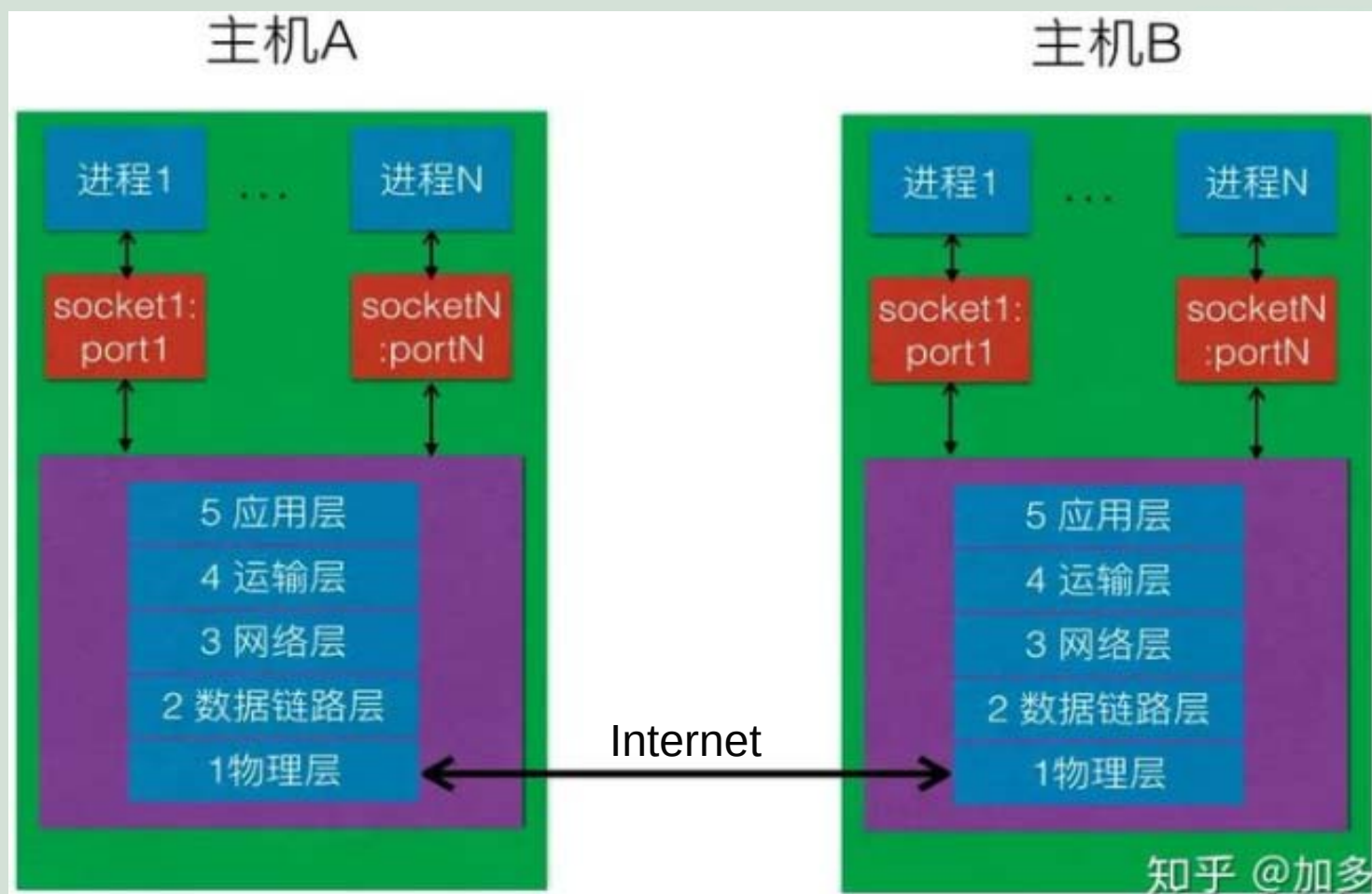
网络编程

- 网络编程最主要的工作是在发送端把信息通过规定好的协议进行组装包，
- 在接收端按照规定好的协议把包进行解析，从而提取出对应的信息，达到通信的目的。
- 中间最主要的就是数据包的组装，数据包的过滤，数据包的捕获，数据包的分析。
- 最后再做一些处理，包括显示、响应等。代码、开发工具、服务器架设和网页设计等可能都会接触。



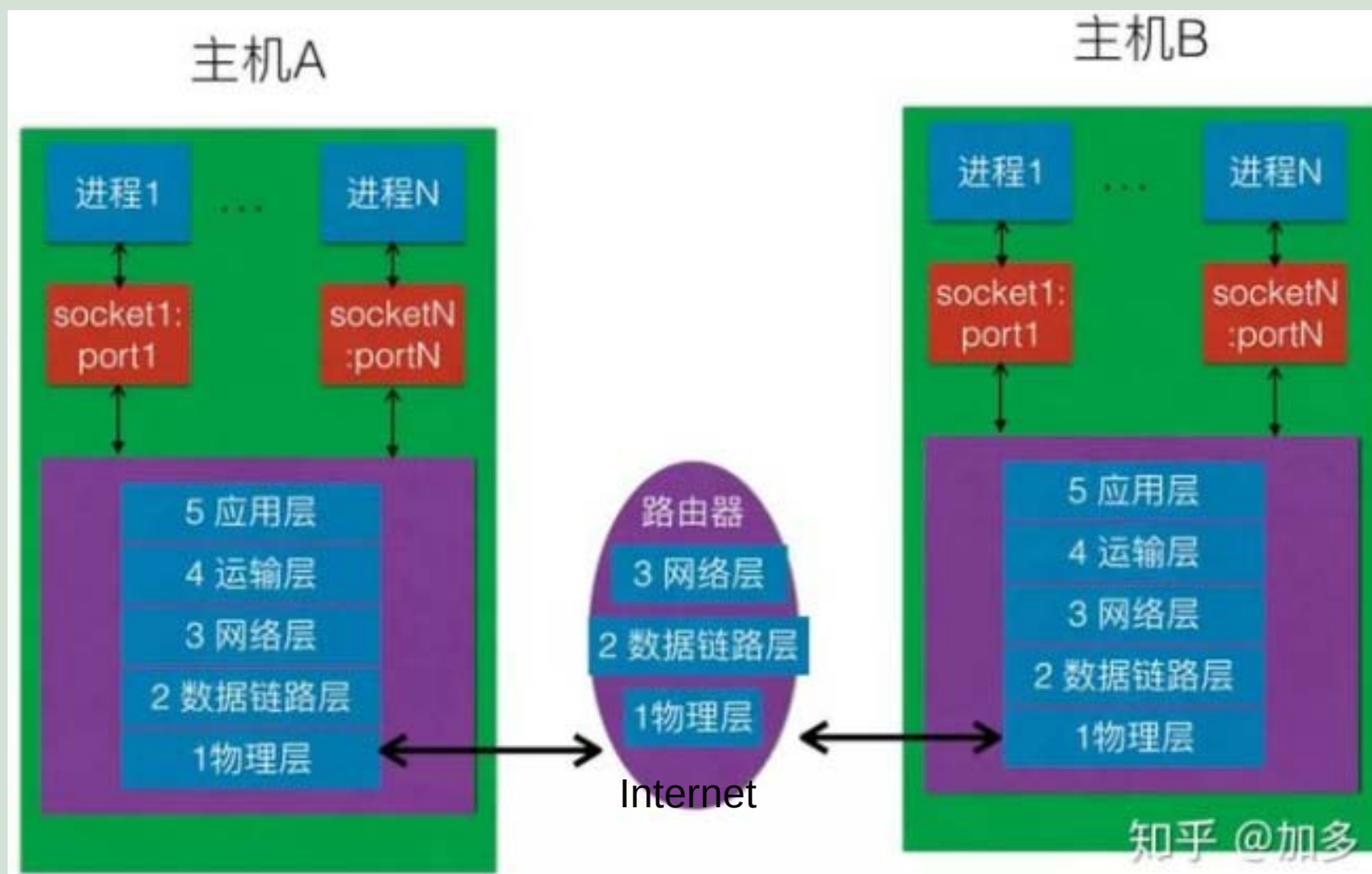
2. 互联网通信技术基础知识

网络应用程序之间网络通讯的本质是网络中处于两个主机上的两个**进程**之间进行通讯。



2. 互联网通信技术基础知识

路由器只有网络层，数据链路层，物理层三层结构。



2. 互联网通信技术基础知识

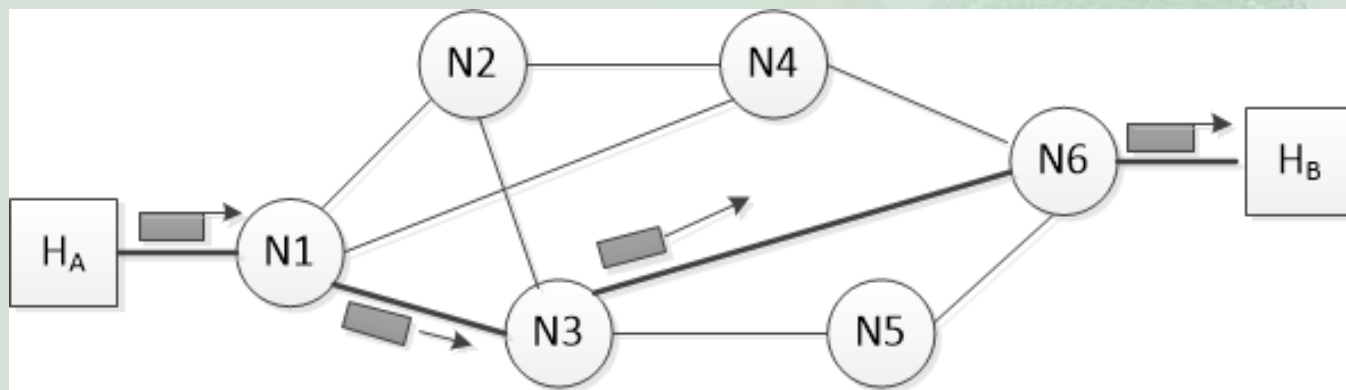
网络的三种交换技术

- 电路交换（线路/电话交换）
- 报文交换
- 分组交换（包交换）



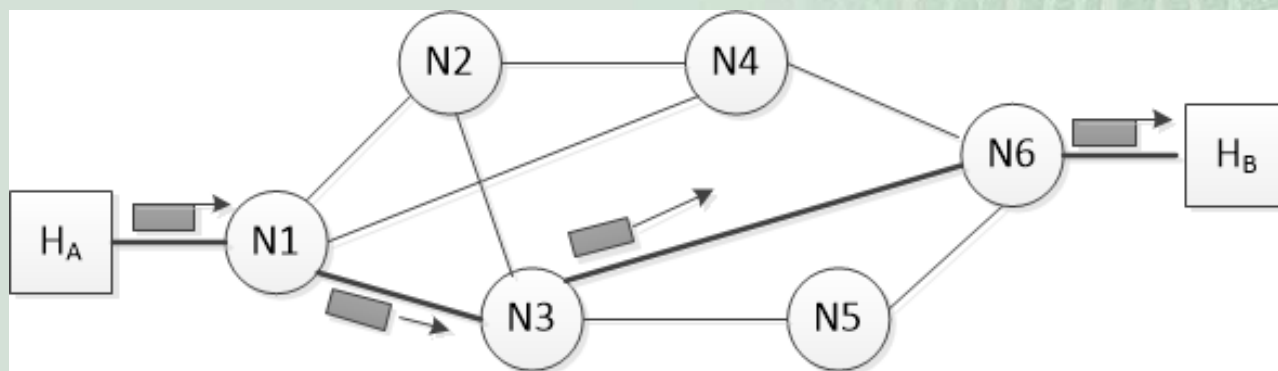
电话交换（线路交换）

- ① 线路建立：发送方提出连接请求，完成逐个结点的接续过程。建立由源站到目的站的传输链路。
- ② 数据传输：全双工传输
- ③ 电路拆除：数据传输结束，由源站（目的站）提出终止通信。各结点拆除相应的连接，释放信道资源。



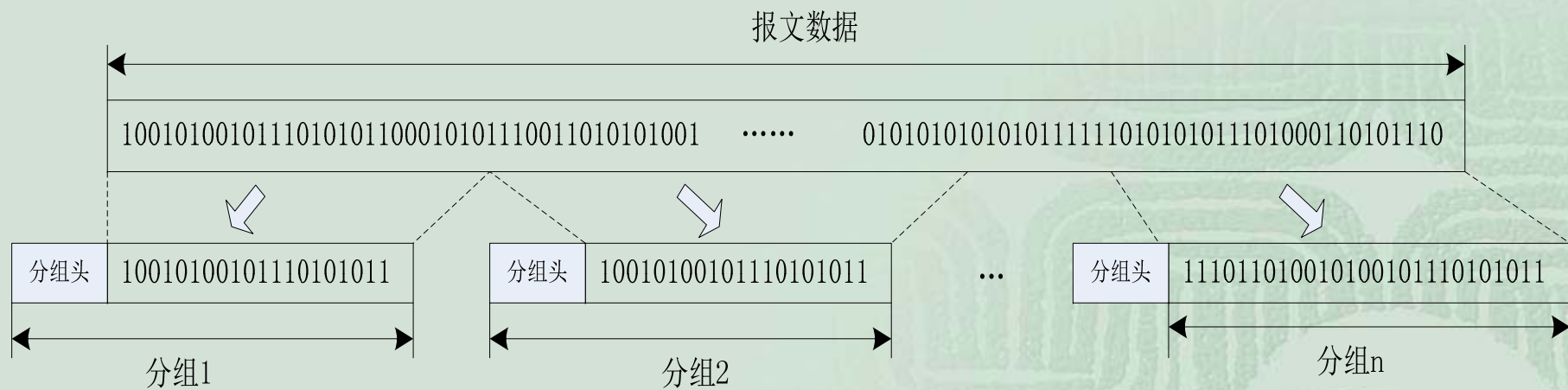
报文交换（存储转发）

- 过程：结点接收一个报文之后，报文缓存（存储设备），根据报文中目的地址转发到下一个结点（如此往复，直到报文到达目的站）。
- 特点：不需要通信双方预先建立专用的数据通路（无需建立链路、拆除链路过程）。
- 缺点：需要对完整的报文进行存储/转发，结点存储/转发的时延较大，不适用于高实时性通信。



交换技术：分组交换

- 属于“存储/转发”交换方式
- 报文被划分为分组，每个分组独立转发

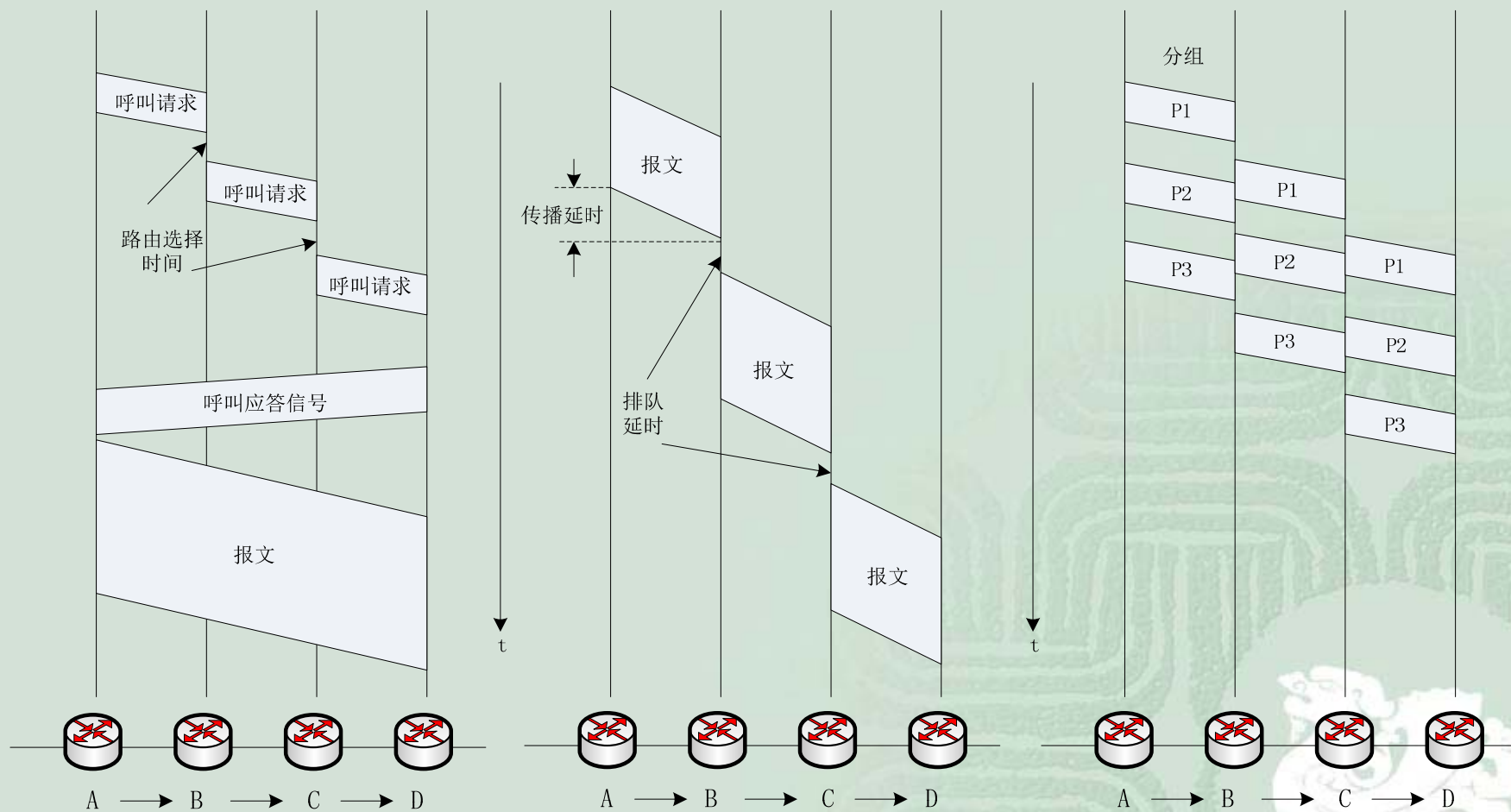


交换技术：分组交换

- 数据报交换：任何分组都当作单独的“小报文”处理，以报文交换方式单独处理分组。
- 虚电路交换：通信双方在开始发送和接收分组之前，需要建立逻辑链路（虚电路）。所有分组都必须沿着事先建立的虚电路传输，需要虚呼叫建立和拆除。

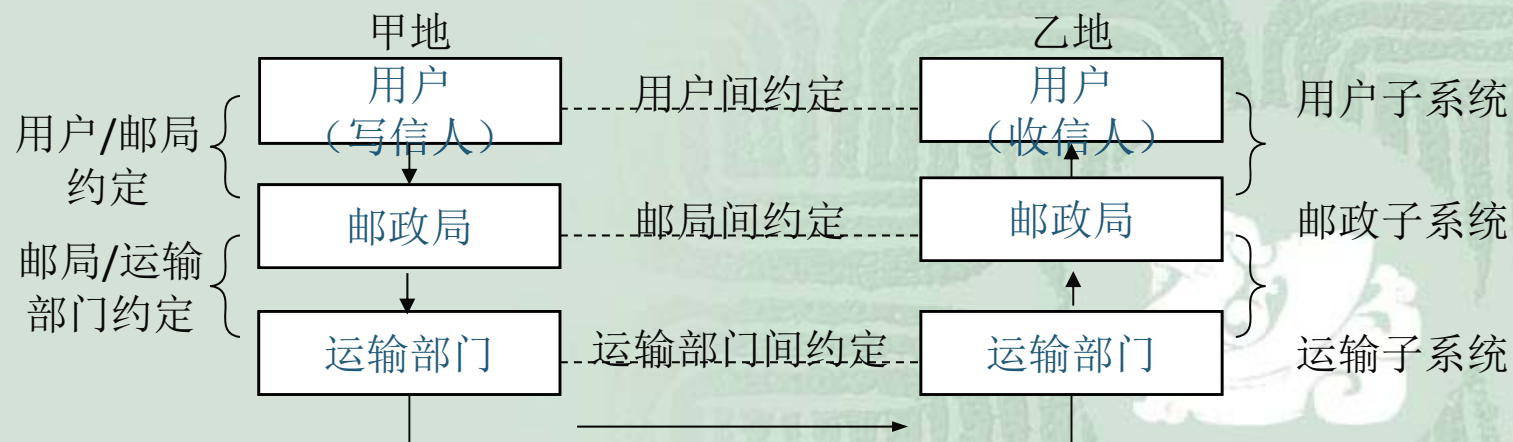


三种交换技术



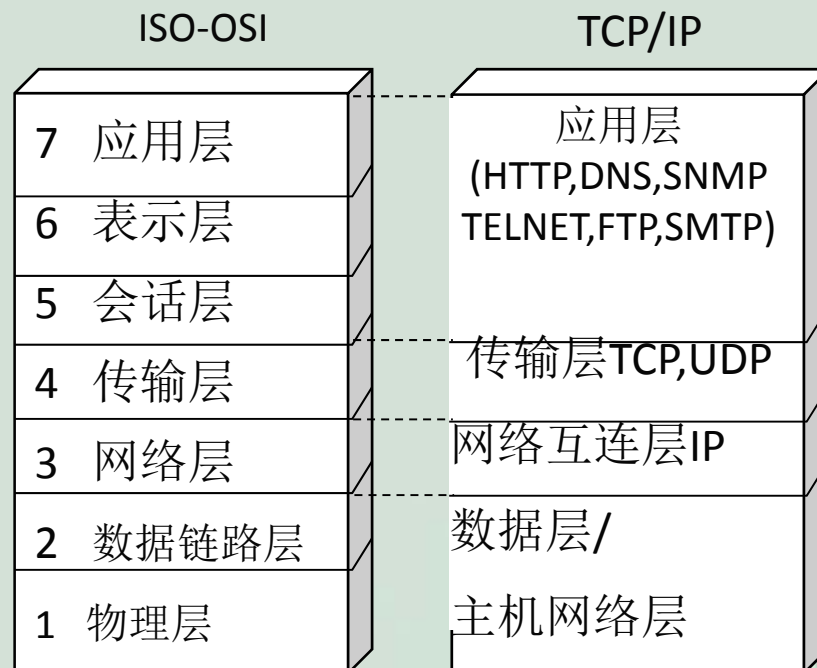
网络协议

- 即采用统一的信息交换规则，规定信息格式，规定如何发送和接收信息
- 网络协议分层
 - (1) 将网络的整体功能分解为功能层，用协议规定功能。
 - (2) 同等功能层之间采用协议进行。
 - (3) 相邻功能层之间采用接口进行交互。

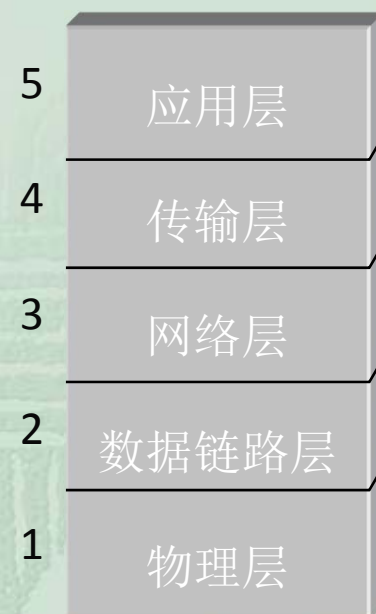


邮政系统分层模型

因特网的体系结构



TCP/IP与ISO-OSI体系结构的对比

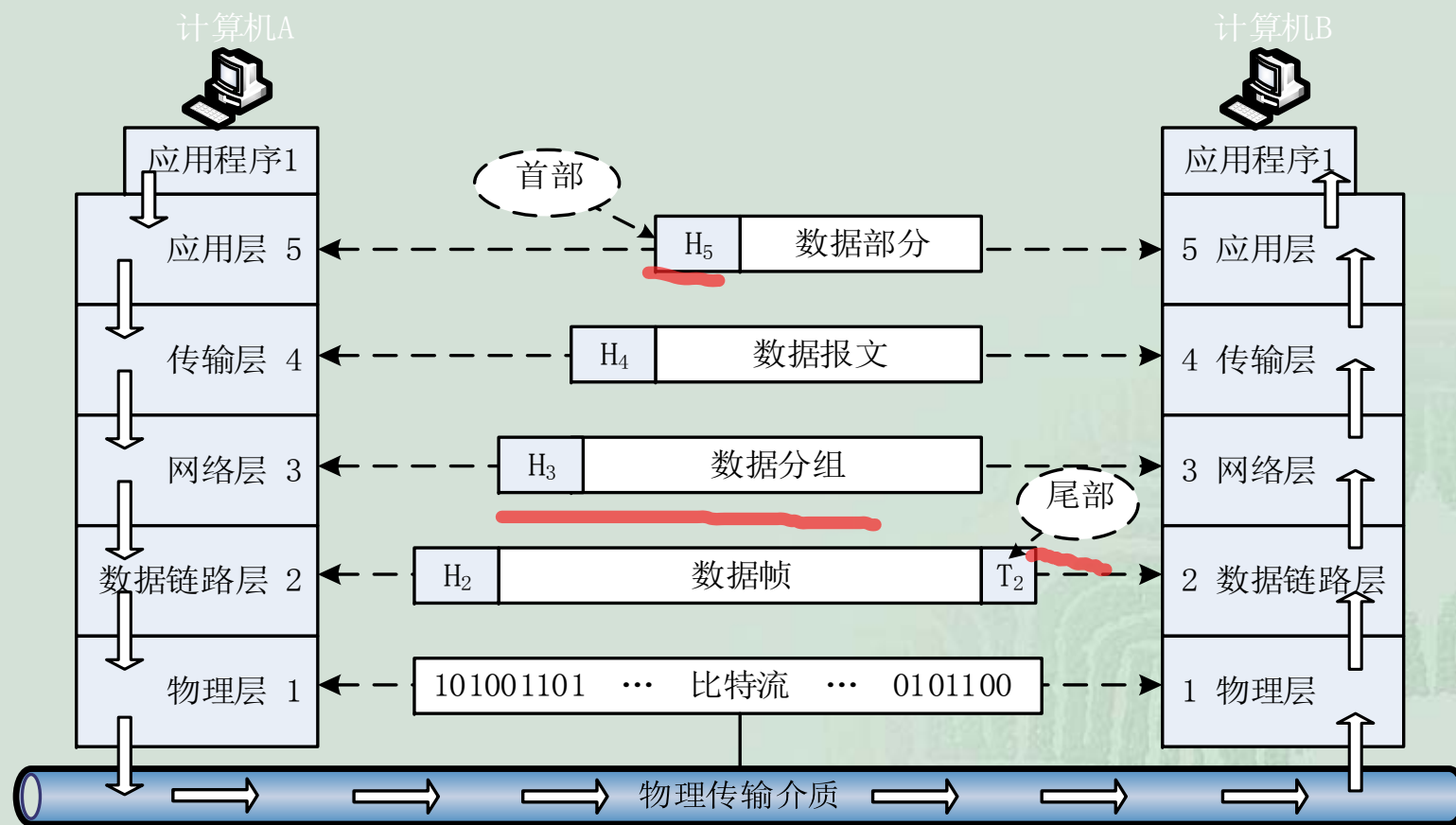


互联网体系结构

因特网的体系结构

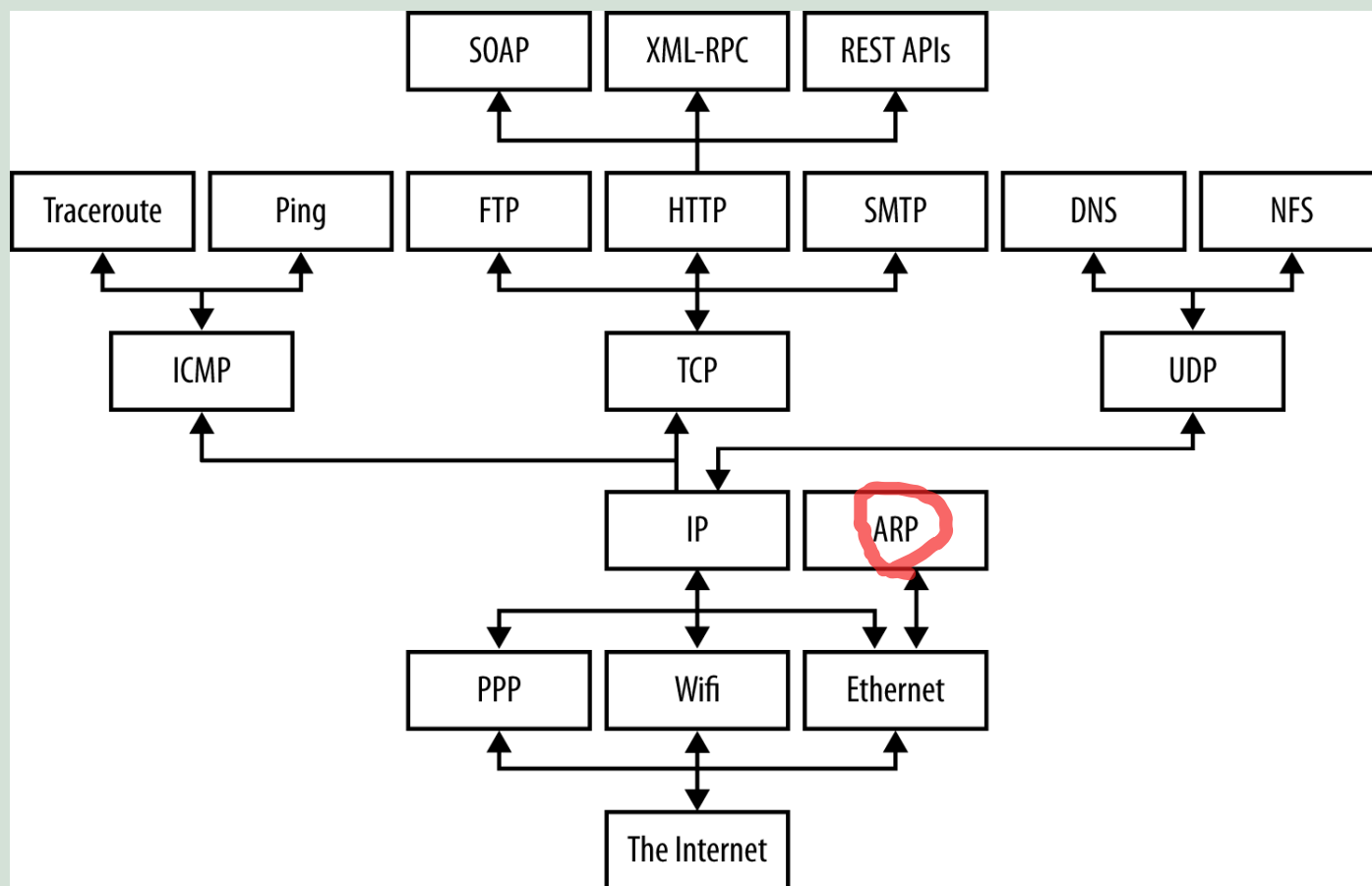
1. 物理层：透明地传送比特流
2. 数据链路层：在两个相邻结点间的线路上无差错地传送以帧为单位的数据。
3. 网络层：完成主机间报文传输，选择合适的路由，使发送方报文能够正确无误地按照地址找到目的站，并交付给目的站。
4. 传输层：进行通信的两个进程之间提供一个可靠的端到端的服务。
5. 应用层：确定进程之间通信的性质，满足用户的需要。

因特网的体系结构



数据在各层之间的传递过程

网络不同层的协议



Internet协议标准化组织

- Internet应用层网络编程和协议的标准大多由IETF和W3C制定。
- IETF (Internet Engineering Task Force, Internet工程任务组), 是不太正式的民间团体, 制定的标准包括TCP/IP、MIME、SMTP。
- **W3C**(World Wide Web Consortium, 国际互联网协会), 是厂商组织, 制定的标准包括HTTP、HTML、XML
- IETF标准和近似标准公布“征求意见稿”(Request for comments, RFC), 一经发布就不再改动。可以修改并允许开发的IETF工作文档称为“Internet草案”。
- W3C标准有5个基本等级, 低→高依次为:
 - ▣ 注解、工作草案、候选推荐、提议推荐、推荐



IP、TCP、UDP

- IP健壮：一个路由器故障，不妨碍网络运行，任意两点之间有多个路由。
- IP平台无关：兼容不同类型计算机
- IP不能保证数据包的到达顺序
- TCP：是面向连接的可靠协议。
- UDP是不可靠协议：不能保证包的顺序，也不能保证一定到达。



IP地址与域名

➤ IPv4: 32 bits

➤ IPv6: 128 bits

➤ 域名与DNS服务

➤ 主机域名转换成IP地址

➤ java.net.InetAddress

URL: http://www.szu.edu.cn

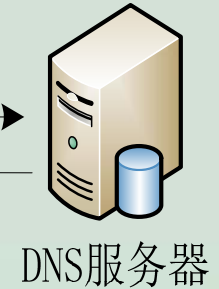
Web浏览器

②浏览器调用DNS客户端程序，并向DNS服务器提出解析深大Web服务器IP地址的请求

③DNS服务器将深大Web服务器的IP地址返回给浏览器

④浏览器根据域名解析得到的IP地址，向深大Web服务器发出HTTP请求

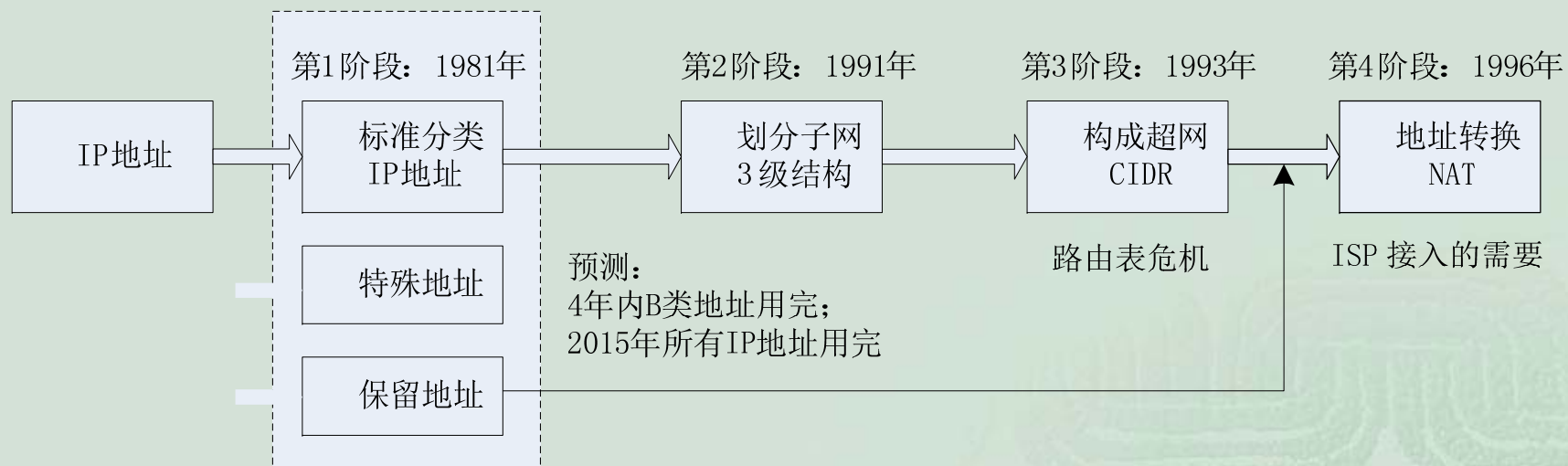
⑤Web服务器将用户需要访问的网页通过HTTP应答返回给浏览器



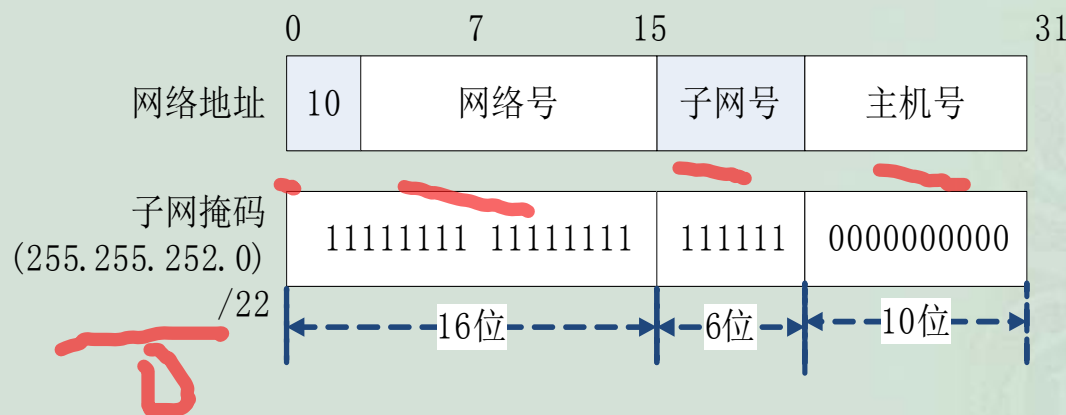
Web服务器



IPv4地址概念与地址划分方法



Network Address Translation, NAT



NAT转换表	
主机专用IP地址	转换后的IP地址
10.0.1.1, 3342	202.0.1.1, 5001
...	...

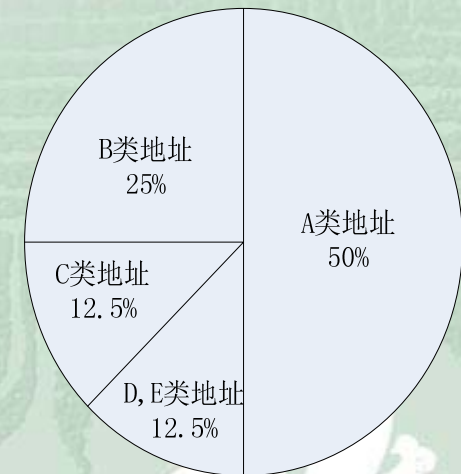
IPv4地址

➤ 地址长度：32位，用点分十进制 (x.x.x.x) 表示 (x=0~255)

➤ 标准IPv4地址的分类：

	1	8	16	24	32	主机地址范围
A类地址	0	网络号 (7位)	主机号 (24位)			1. 0. 0. 0 – 127. 255. 255. 255
B类地址	10	网络号 (14位)		主机号 (16位)		128. 0. 0. 0 – 191. 255. 255. 255
C类地址	110	网络号 (21位)			主机号 (8位)	192. 0. 0. 0 – 223. 255. 255. 255
D类地址	1110	网络号 (28位)				224. 0. 0. 0 – 239. 255. 255. 255
E类地址	11110	保留用于实验和将来使用				240. 0. 0. 0 – 247. 255. 255. 255

a) 标准分类的IP地址



b) 每类地址比例数量

特殊IP地址

网络标识	主机标识	意 义
全0	全0	代表本网络上的本主机
全0	主机号	代表本网络上的某个主机
网络号	全0	代表指定的一个网络
全1	全1	只限本网络上进行广播 (受限广播地址)
网络号	全1	对网络号上所有主机进行广播 (直接广播地址)
127		用作本地循环测试 (loopback test)

专用IP地址

类别	网络号	总 数
A	10	1
B	172.16 ~ 172.31	16
C	192.168.0 ~ 192.168.255	256

IPv6前缀格式

- IPv6不支持子网掩码，它只支持前缀长度表示法。
- 前缀是IPv6地址的一部分，用作IPv6路由或子网标识。

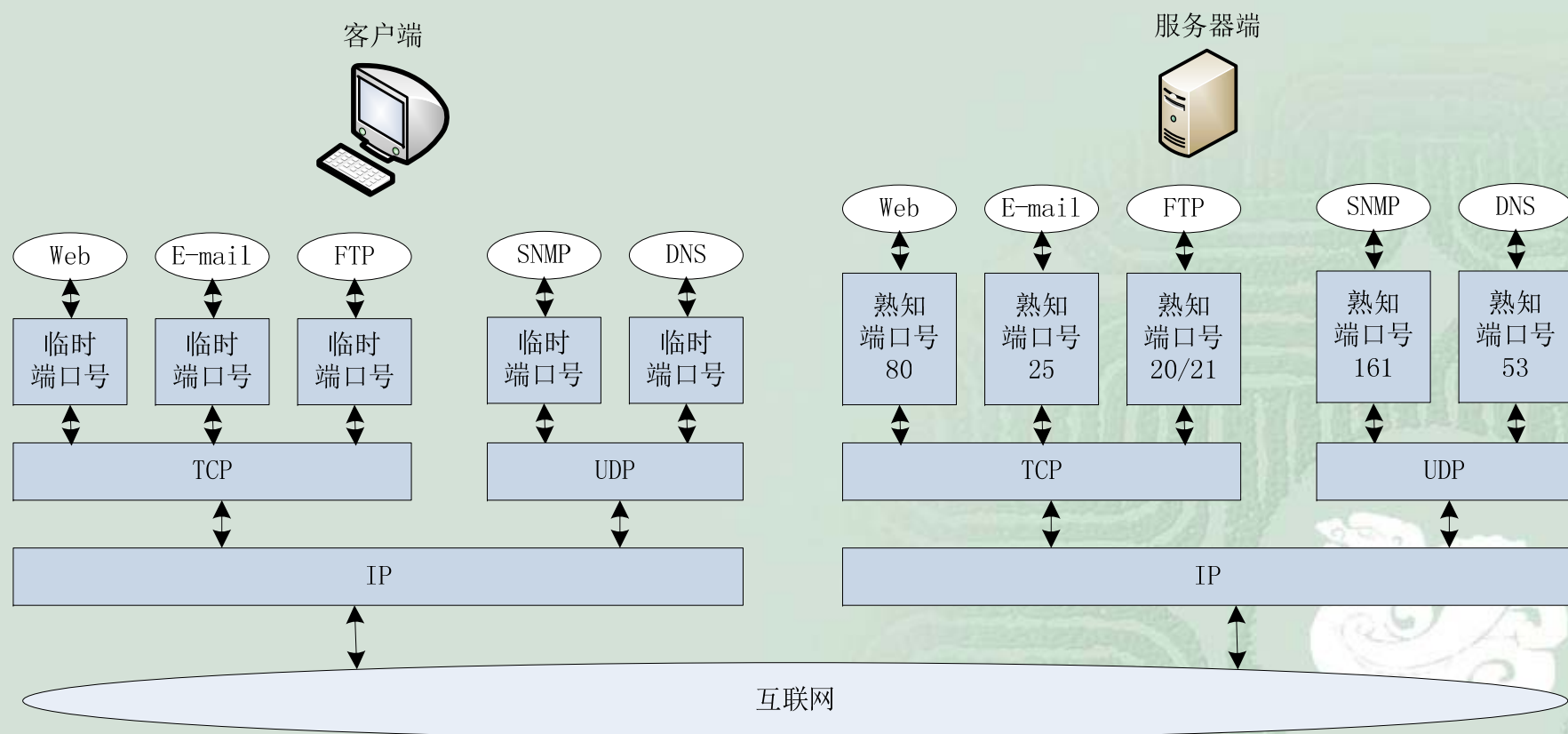
格式： IPv6地址/前缀长度n

地址类型	IPv6前缀表示	含义
未指定	::/128	表示不存在的地址，不能被分配给任意节点
环回地址	::1/28	表示节点自身，不能被分配给任意物理接口
组播地址	FF00::/8	表示发送给某个组播地址
链路本地地址	FE80::/10	表示仅在本地铁路上使用的地址，用于地址配置等场景
站点本地地址	FEC0::/10	表示仅用于站点内部的地址，已废除
全球单播地址	其他所有	表示可被分配到公网的地址，是全球可路由的

端口/Port

IP作用是作为主机的标识，端口port作用是标定主机中的进程。

- 解决多个网络应用程序同时工作问题, 应用进程标识的基本方法
- 传输层进程寻址：通过TCP/UDP端口号实现



端口号的分配

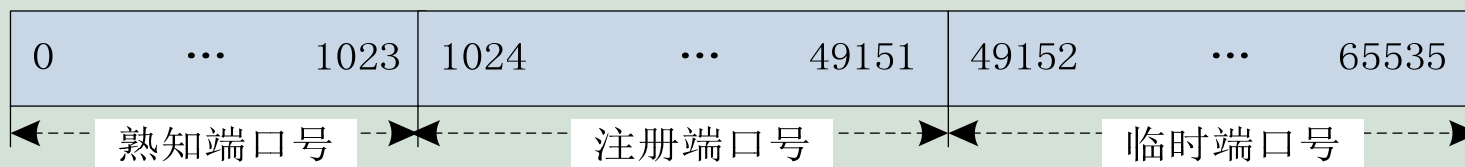


表1 UDP常用的熟知端口号

端口号	服务进程	说 明
53	Domain	域名服务
67/68	DHCP	动态主机配置协议
69	TFTP	简单文件传输协议
111	RPC	远程过程调用
123	NTP	网络时间协议
161/162	SNMP	简单网络管理协议
520	RIP	路由信息协议

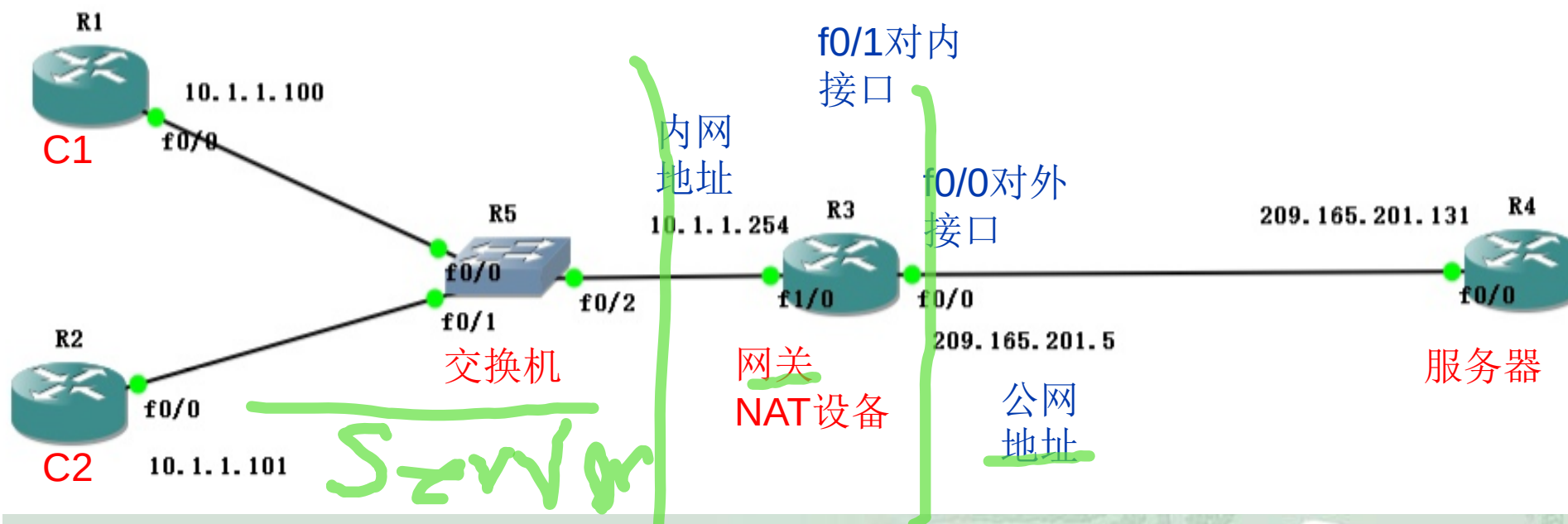
表2 TCP常用的熟知端口号

端口号	服务进程	说 明
20	FTP	文件传输（数据连接）
21	FTP	文件传输（控制连接）
23	TELNET	网络虚拟终端协议
25	SMTP	简单邮件传输协议
80	HTTP	超文本传输协议
119	NNTP	网络新闻传输协议
179	BGP	边界路由协议

NAT转换

用户C1向服务器S发起一个HTTP请求，经过NAT转化，服务器收到并作出响应，用户C1收到响应。

问题:NAT是怎么知道这个响应是给用户C1而不是C2的呢?

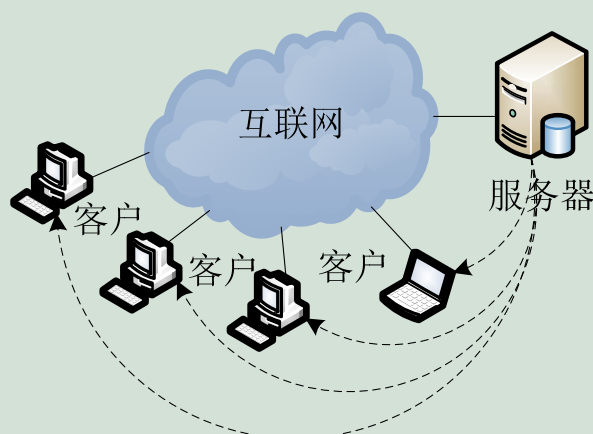


Network Address Translation, NAT

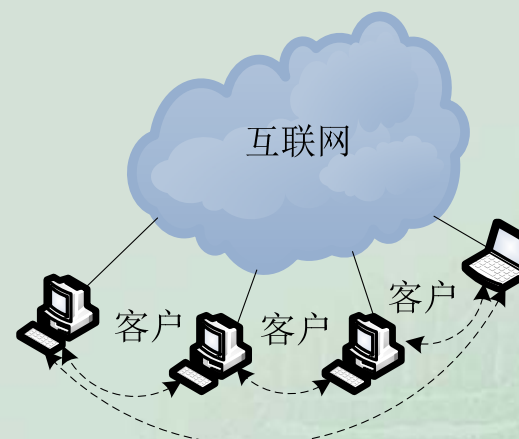


3. 互联网通信应用程序

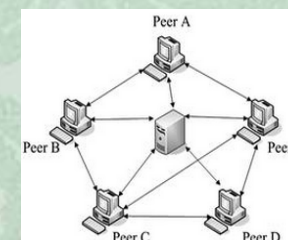
互联网通信应用程序的常见工作模式



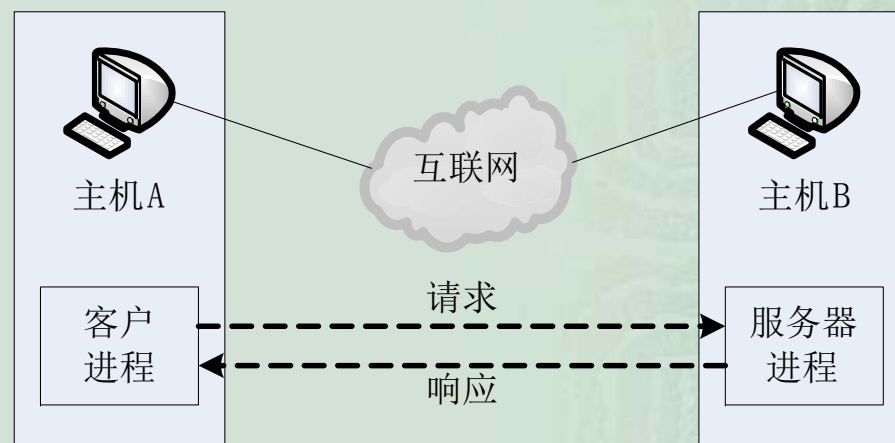
a) C/S工作模式



b) P2P工作模式



- 服务器程序在固定的IP地址和熟知的端口号上一直处于打开状态
- 客户端之间不能够直接通信



➤ P2P网络并不是一个新的网络结构，而是一种新的网络应用模式

套接字Socket通信编程

socket

- ◆ 端口号与IP地址的组合得出一个网络套接字。
- ◆ 端口号被规定为一个16位的整数0~65535。
- ◆ 0~1023被预先定义的服务通信占用（如telnet占用端口23，http占用端口80等）。
- ◆ 除非我们需要访问这些特定的服务，否则，就应该使用1024~65535这些端口中的某一个进行通信，以免发生端口冲突。



套接字Socket通信编程

- ◆ 1.套接字连接
- ◆ 套接字连接就是客户端的套接字对象和服务器端的套接字对象通过输入、输出流连接在一起，现在我们分三个步骤来说明套接字连接的基本模式。
- ◆ (1) 服务器建立ServerSocket对象
- ◆ ServerSocket对象负责等待客户端的请求，进而建立套接字连接。
- ◆ ServerSocket的构造方法是：ServerSocket(int port)

```
try
{
    ServerSocket waitSocketConnection = new ServerSocket(1880);
}
catch(IOException e){}
```

套接字Socket通信编程

- 当服务器端的ServerSocket对象waitSocketConnection建立后，就可以使用方法accept()接收客户端的套接字连接请求，代码如下所示：

```
Socket socketAtServer = waitSocketConnection.accept();
```

- 所谓“接收”客户的套接字请求，就是accept()方法会返回一个Socket对象（即socketAtServer），称作服务器端的套接字对象。



每次只能接收一个，若要接收多个客户端就要多次调用accept()方法

套接字Socket通信编程

■ (2) 客户端创建Socket对象

- 客户端程序可以使用Socket类创建对象，Socket的构造方法是：Socket(String host, int port)

```
Socket socketAtClient = new Socket(localhost, 1880);
```

- 2 ■ 也可以使用Socket类不带参数的构造方法public Socket()

- 该对象再调用public void connect(InetSocketAddress endpoint) throws IOException **请求**和参数InetSocketAddress指定地址的套接字建立连接。

- 客户端建立Socket对象（即socketAtClient）的过程就是向服务器发出套接字连接请求，如果服务器端相应的端口上有套接字对象正在使用accept()方法等待客户，那么双方的套接字对象（即socketAtClient和socketAtServer）就都诞生了。

套接字Socket通信编程

- **(3) 流连接**
- 客户端和服务端端的**Socket对象**诞生以后，还必须进行输入、输出流的连接。
- 服务器端的这个Socket对象（即socketAtServer）使用方法getOutputStream()获得的**输出流**，~~将~~指向客户端Socket对象（即socketAtClient）使用方法getInputStream()获得的那个**输入流**。
- 服务器端的这个Socket对象（即socketAtServer）使用方法getInputStream()获得的**输入流**，~~将~~指向客户端Socket对象（即socketAtClient）使用方法getOutputStream()获得的那个**输出流**。
- 因此，当服务器向这个输出流写入信息时，客户端通过相应的输入流就能**读取**，反之亦然。

套接字Socket通信编程

- 【例子】
- Example11_5
 - Client.java
 - Server.java



套接字Socket通信编程例子

```
import java.io.*;
import java.net.*;
public class Server
{
    public static void main(String args[])
    {
        ServerSocket server = null;
        Socket socketAtServer = null;
        DataOutputStream out = null;
        DataInputStream in = null;

        try
        {
            server = new ServerSocket(4333);
        }
        catch(IOException e1)
        {
            System.out.println("ERROR:"+e1);
        }
    }
}
```

套接字Socket通信编程例子

```
try
{
    socketAtServer = server.accept();
    in = new DataInputStream(socketAtServer.getInputStream());
    out = new DataOutputStream(socketAtServer.getOutputStream());
    while(true)
    {
        int m = 0;
        m = in.readInt();
        out.writeInt(m*2);
        System.out.println("Server received: " + m);
        Thread.sleep(500);
    }
}
catch(IOException e)
{
    System.out.println(""+e);
}
catch(InterruptedException e){}
}
```

套

```
import java.io.*;
import java.net.*;
public class Client
{
    public static void main(String args[])
    {
        Socket socketAtClient;
        DataInputStream in = null;
        DataOutputStream out = null;
        try
        {
            socketAtClient = new Socket("localhost", 4333);
            in = new DataInputStream(socketAtClient.getInputStream());
            out = new DataOutputStream(socketAtClient.getOutputStream());
            out.writeInt(1);
            while(true)
            {
                int m2 = 0;
                m2 = in.readInt();
                out.writeInt(m2);
                System.out.println("Client received: " + m2);
                Thread.sleep(500);
            }
        }
        catch(IOException e)
        {
            System.out.println("Unable to connect to the server");
        }
        catch(InterruptedException e){}
    }
}
```


套接字Socket通信编程

- 先运行服务器端程序Server.java
- 再运行客户端程序Client.java

```
C:\Windows\system32\cmd.exe - java Server

E:\WorkspaceEclipse\Example11_5\bin>java Server
Server received: 1
Server received: 2
Server received: 4
Server received: 8
Server received: 16
Server received: 32
Server received: 64
Server received: 128
Server received: 256
Server received: 512
Server received: 1024
Server received: 2048
Server received: 4096
Server received: 8192
Server received: 16384
Server received: 32768
Server received: 65536
Server received: 131072
Server received: 262144
Server received: 524288
Server received: 1048576
Server received: 2097152
```

```
C:\Windows\system32\cmd.exe - java Client

E:\WorkspaceEclipse\Example11_5\bin>java Client
Client received: 2
Client received: 4
Client received: 8
Client received: 16
Client received: 32
Client received: 64
Client received: 128
Client received: 256
Client received: 512
Client received: 1024
Client received: 2048
Client received: 4096
Client received: 8192
Client received: 16384
Client received: 32768
Client received: 65536
Client received: 131072
Client received: 262144
Client received: 524288
Client received: 1048576
Client received: 2097152
Client received: 4194304
```



谢谢

小结重、难点

1. IP表示、端口
2. 互联网通信应用程序的结构

